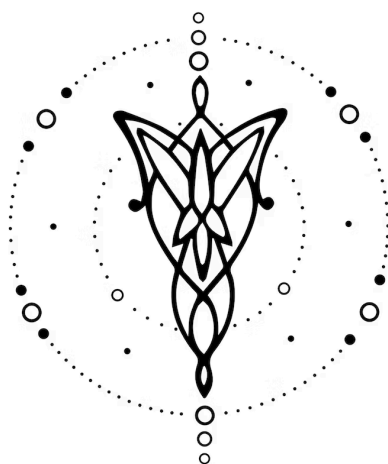




Especificación - DGF

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Catalina	Trajterman	64175	ctrajterman@itba.edu.ar
María Belén	Petrikovich	63689	mpetrikovich@itba.edu.ar
Luana	Percich	64316	lpercich@itba.edu.ar

2. Repositorio

La solución de este trabajo práctico será versionada en: [DELTA-GIRLS-FORCE](https://github.com/DELTA-GIRLS/DELTA-GIRLS-FORCE)

3. Dominio

El dominio para este proyecto es diseñar un programa para convertir una expresión del álgebra relacional en formato JSON a su expresión equivalente en SQL.

La implementación de este proyecto permitirá observar la correlación entre el lenguaje teórico y el que se usa en la práctica.

4. Construcciones

El lenguaje propuesto tendrá las siguientes construcciones, prestaciones y funcionalidades:

1. Cada consulta se representará como un objeto JSON, con campos como "operation", "attributes", "input", "name" y "condition". Estos campos dependen de la operación realizada.
2. Se admitirán los principales operadores del álgebra relacional (selección, proyección, join, etc.), cada uno con su representación en JSON y su traducción a SQL.
3. El campo "condition" permitirá expresar condiciones compuestas utilizando conectores, y será traducido con la cláusula "WHERE".
4. Cada construcción definida en el JSON tiene su correlato directo en SQL, garantizando que la expresión de álgebra relacional representada en este lenguaje pueda ser compilada a una consulta SQL funcionalmente equivalente.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

1. Un programa que realice una proyección mostrando únicamente los atributos indicados de una relación.
2. Un programa que permita renombrar una relación o un atributo y use ese nombre en la salida SQL (operación rho)
3. Un programa que realice una selección con condiciones dentro de {<=, >=, >, <, =, !=}
4. Un programa que realice un join e indique condición y qué tabla va por izquierda y cual por derecha.
5. Un programa que utilice en una operación binaria diferencia (-).
6. Un programa que contenga consultas anidadas.
7. Un programa que realice el producto cartesiano entre dos relaciones.
8. Un programa que ejecute una unión de dos relaciones.
9. Un programa que ejecute una intersección de dos relaciones.
10. Un programa que combine selección y proyección de manera anidada.

Además, los siguientes casos de prueba de **rechazo**:

1. Un programa mal formado.
2. Un programa con una operación inválida.
3. Un programa que contenga input vacío o inválido.
4. Un programa que pida una condición inválida.
5. Un programa que no contenga un atributo que es obligatorio.
6. Un programa con una operación de outer o inner join

6. Ejemplos

1. Entrada:

```
{
  "operation": "projection",
  "attributes": ["nombre"],
  "input": {
    "operation": "selection",
    "condition": "edad > 18",
    "input": {
      "operation": "table",
      "name": "Persona"
    }
  }
}
```

Salida:

```
SELECT nombre FROM Persona WHERE edad > 18;
```

2. Entrada:

```
{
  "operation": "projection",
  "attributes": ["Persona.nombre", "Cursa.curso"],
  "input": {
    "operation": "join",
    "condition": "Persona.legajo = Cursa.legajo",
    "left": {"operation": "table", "name": "Persona"},
    "right": {"operation": "table", "name": "Cursa"}
  }
}
```

Salida:

```
SELECT Persona.nombre, Cursa.curso
FROM Persona
JOIN Cursa ON Persona.legajo = Cursa.legajo;
```